

Manual de aprendizaje

De Kotlin y Android a React: todo lo que implementa el proyecto
Bodymetria · 12/07/2026

Guía para entender TODO lo que usa la app, desde cero, con ejemplos tomados del propio código.
Orden recomendado de lectura: es un curso, no una referencia.

1. El panorama: ¿qué es una app de Android?

Una app Android es un paquete (APK) que contiene código compilado, recursos (iconos, textos) y un manifiesto (`AndroidManifest.xml`) que declara qué contiene la app y qué permisos pide. Bodymetria no pide ninguno.

Conceptos base:

- **Activity**: una "ventana" de la app. Bodymetria tiene UNA sola (`MainActivity`) y toda la navegación ocurre dentro, dibujada por Compose.
- **Sandbox**: cada app tiene su carpeta privada que ninguna otra app puede leer. Ahí vive nuestra base de datos. Por eso el respaldo JSON existe: es la única forma de sacar los datos.
- **APK vs AAB**: el APK se instala directo (lo que usamos); el AAB es el formato que exige Play Store para publicar.
- **minSdk / targetSdk**: la versión mínima de Android que aceptamos (26 = Android 8.0) y la versión contra la que declaramos comportarnos bien (35).

2. Kotlin: el lenguaje

Kotlin es el lenguaje oficial de Android: conciso, con tipos estáticos y diseñado para eliminar el error más común de Java (el null inesperado).

- **val / var**: inmutable / mutable. Preferimos `val`.
- **Null safety**: `Float?` puede ser null, `Float` no. El compilador te OBLIGA a manejarlo:
`musculoKg?.let { ... }` ejecuta el bloque solo si no es null; `?:` da un valor por defecto (`texto.toFloatOrNull() ?: 0f`).
- **data class**: clase que solo guarda datos; equals/copy gratis. `data class RegistroMacros(val fecha: String, val proteinasG: Float, ...)`.
- **Lambdas**: funciones anónimas entre llaves. `historial.filter { it.fecha >= hace7 }` filtra una lista; `it` es el elemento actual.
- **Funciones de extensión**: añadir métodos a tipos existentes; Compose las usa por todos lados (`Modifier.fillMaxWidth()`).
- **when**: un switch potente que devuelve valor (así elegimos los parámetros de meta según el objetivo en `Metas.kt`).
- **Corrutinas**: concurrencia ligera. Una función `suspend` puede "pausarse" sin bloquear la interfaz; así escribimos a la base de datos sin congelar la pantalla. Se lanzan con `ambito.launch { db.macros().guardar(r) }`.
- **Flow**: un flujo de valores en el tiempo. Room devuelve `Flow<List<...>>`: cada vez que la tabla cambia, emite la lista nueva y la UI se repinta sola.

3. Jetpack Compose: la interfaz

Compose es el sistema de UI declarativo de Android: en vez de editar vistas, DESCRIBES la pantalla como funciones y el framework la redibuja cuando cambia el estado.

- **@Composable**: marca una función que dibuja UI. Se componen unas dentro de otras: `Column { Text("Hola"); Button(...) }`.
- **Estado**: `var peso by remember { mutableStateOf("") }`. Cuando peso cambia, TODO lo que lo lee se recompone (redibuja) automáticamente. `remember` conserva el valor entre recomposiciones.
- **Recomposición**: el ciclo de vida de Compose. No mutamos vistas: mutamos estado y Compose recalcula la pantalla.
- **collectAsState**: puente Flow-estado. La lista del historial se actualiza sola al guardar o borrar, sin código extra.
- **LaunchedEffect(clave)**: corre una corrutina cuando la clave cambia; lo usamos para rellenar el formulario al cambiar de fecha y para recalculr el gasto estimado al cambiar disciplina o duración.
- **Modifier**: cadena de ajustes de layout/estilo: `Modifier.fillMaxWidth().padding(16.dp)`.
- **Material 3**: componentes con el lenguaje visual de Google (Card, Button, Slider, AlertDialog, FilterChip...). En Android 12+ la paleta se genera del fondo de pantalla del usuario ("Material You"); por eso los colores de la app cambian según el dispositivo.
- **Canvas**: lienzo de dibujo libre. Nuestras gráficas son un Canvas donde trazamos rejilla, línea/barras y etiquetas a mano (sin librerías: menos peso y control total).
- **Navigation Compose**: mapa de rutas a composables. `NavHost` declara `composable("macros") { PantallaMacros(nav) }` y las pantallas navegan con `nav.navigate("macros")`. La ruta `diario/{tipo}` recibe un parámetro: UNA pantalla genérica sirve para estrés, agua, lectura, meditación, ánimo y regla.

4. Room: la base de datos

Room es una capa fina sobre SQLite (la base de datos embebida estándar).

- **@Entity**: una tabla. La clase `RegistroMacros` ES la fila.
- **@Dao**: interfaz con las consultas. Room GENERA la implementación: `@Query("SELECT * FROM macros WHERE fecha = :fecha")`.
- **@Upsert**: inserta o actualiza según la clave primaria (así "guardar" el mismo día sobrescribe en vez de duplicar).
- **@Relation / @Transaction**: consultas anidadas; así `SesionCompleta` trae la sesión con sus ejercicios y series en un solo objeto.
- **Flow en consultas**: la magia reactiva — la UI observa la tabla.
- **Migraciones**: cuando el esquema cambia entre versiones hay que decirle a Room CÓMO transformar la base vieja sin borrar datos (v1->v2 fue un `ALTER TABLE diario ADD COLUMN texto`). Sin migración la app truena, o peor, borra todo si usas `fallbackToDestructiveMigration` (prohibido aquí).
- **KSP**: el procesador que genera el código de Room al compilar.

Decisión de diseño que vale estudiar: la tabla `diario(fecha, tipo, valor, texto)` es un patrón "entidad-atributo-valor" acotado — permite añadir secciones nuevas (micros `micro.fibra`, métricas

médicas med.3) SIN tocar el esquema. El costo: menos validación por columna; lo aceptamos porque los tipos los controla el código.

5. DataStore: las preferencias

Para datos pequeños clave-valor (el perfil) usamos DataStore Preferences en lugar de una tabla: API de Flow, escritura atómica y sin SQL. `flujoPerfil` emite el `Perfil` cada vez que cambia.

6. kotlinx.serialization: el respaldo JSON

`@Serializable` sobre las data classes + `Json.encodeToString` produce el respaldo completo. Al importar: `ignoreUnknownKeys` tolera campos futuros y los valores con default toleran respaldos viejos (compatibilidad en ambos sentidos). La lección de seguridad: NUNCA aplicar datos externos sin validar (rangos, fechas, tamaños) y sin confirmación del usuario.

7. Gradle: el sistema de construcción

Gradle compila, resuelve dependencias y empaqueta el APK.

- **gradlew (wrapper)**: script que descarga la versión EXACTA de Gradle del proyecto; por eso no se instala Gradle a mano.
- **build.gradle.kts**: configuración en Kotlin. El del módulo app declara `compileSdk/minSdk`, plugins y dependencias.
- **libs.versions.toml**: catálogo central de versiones (un solo lugar para actualizar).
- **AGP**: el plugin de Android para Gradle (convierte recursos, firma, dexa).
- **versionCode / versionName**: contador interno (debe subir en cada release de Play Store) y versión visible ("0.1").
- **debug vs release**: debug se firma con una llave de prueba automática; release exige TU keystore (y esa llave hay que resguardarla: sin ella no hay actualizaciones futuras en Play Store).

8. La cadena de herramientas sin Android Studio

Esta app se construyó SIN Android Studio, para entender las piezas:

- **JDK 17**: Java Development Kit; Kotlin compila sobre la JVM.
- **Android SDK**: `platform-tools` (adb), `platforms/android-35` (android.jar) y `build-tools` (aapt2, firmador). Se instaló con `cmdline-tools`.
- **adb**: puente con el dispositivo/emulador: `adb install -r app.apk`, `adb shell screencap`, `adb logcat`. Es la navaja suiza.
- **Emulador**: un Android virtual (AVD). Creamos "bodymetria" (Pixel 6, Android 15) con `avdmanager`; se abre con `Abrir Bodymetria.bat`.
- Trampa de ESTA red: TLS interceptado — la JVM necesita `javax.net.ssl.trustStoreType=Windows-ROOT` (ya fijado en `gradle.properties`) y `curl --ssl-no-revoke`. Si algo no descarga, casi seguro es esto.

- Trampa de ESTE entorno: algunas apps de escritorio empaquetadas (MSIX) virtualizan las escrituras en AppData. Lo instalado ahí puede quedar en una carpeta LocalCache del paquete e invisible para el resto del sistema; se resolvió moviendo el SDK al disco real.

9. Los algoritmos de la app

- **Mifflin-St Jeor** (metas de kcal): ecuación de metabolismo basal validada: $10 \cdot \text{peso} + 6.25 \cdot \text{estatura} - 5 \cdot \text{edad}$ más una constante por sexo. La multiplicamos por 1.3 (actividad cotidiana) y le sumamos la media real de gasto por ejercicio de los últimos 7 días — el factor de actividad sale de TUS datos, no de una tabla genérica.
- **Ajustes por objetivo**: déficit 20% (pérdida), superávit 10% (ganancia), déficit 8% con proteína de énfasis vegetal (longevidad). Proteína por kg y grasa como % de kcal; carbohidratos = resto.
- **METs** (gasto por ejercicio): 1 MET = 1 kcal/kg/hora en reposo. Cada disciplina tiene su MET aproximado (correr 9.8, fútbol 7, caminata 3.5...); kcal = MET x peso x horas. Es la base del Compendium of Physical Activities y de casi todos los relojes deportivos.
- **Estadística**: media (ventanas de 7/30 días), varianza poblacional (promedio de las desviaciones al cuadrado) y desviación estándar (su raíz), tendencia = media(últimos 7) – media(7 anteriores). La varianza mide qué tan CONSISTENTE eres, no solo el promedio: dos personas con media de 7 h de sueño viven muy distinto si una duerme 7 ± 0.3 h y la otra 7 ± 2 h.
- **RDA de micros**: valores de referencia (NIH) por sexo; el sodio se trata como tope máximo en vez de mínimo.

10. Seguridad y privacidad aplicadas

- Sandbox de Android: la base es ilegible para otras apps sin root.
- Cero permisos y cero red: no hay superficie de ataque remota.
- Storage Access Framework para exportar/importar: la app nunca ve el sistema de archivos completo, solo el archivo que el usuario eligió.
- Import validado + vista previa: los datos externos son entrada hostil hasta demostrar lo contrario.
- Borrados con confirmación en toda la app.

11. El sitio web: React, Vite y localStorage

El sitio (web/) es la otra mitad del proyecto: complementa a la app con la base de alimentos, el menú y las guías. Comparte la filosofía: **todo local, sin servidor**. Piezas clave:

- **React**: librería de UI declarativa (el equivalente web de Compose). La interfaz se describe con componentes que devuelven "JSX" (HTML dentro de JavaScript); cuando el estado cambia, React re-renderiza solo lo necesario.
- **Componentes y props**: funciones que devuelven UI y reciben datos por parámetros. Cada página (Alimentos, Recetas, Micros...) es un componente.
- **useState / useMemo / useEffect**: los "hooks". useState guarda estado que al cambiar redibuja (como useStateOf en Compose); useMemo memoriza cálculos caros; useEffect corre efectos

(p. ej. guardar en `localStorage`).

- **localStorage**: el "Room" del navegador — un almacén clave-valor por sitio. Ahí viven el perfil y el registro del día; nada sale del navegador.
- **Vite**: el empaquetador/servidor de desarrollo (el "Gradle" del front): `npm run dev` levanta el sitio con recarga en caliente; `npm run build` genera la carpeta `dist/` estática.
- **React Router (HashRouter)**: navegación por rutas del lado del cliente; el hash (`#/recetas`) evita necesitar configuración de servidor.
- **El cálculo nutricional**: las recetas no guardan calorías "a mano" — cada ingrediente referencia la base (`[idAlimento, raciones]`) y `kcal/macros/micros` se SUMAN de ahí. Así, corregir un alimento arregla todas las recetas que lo usan (una sola fuente de verdad, el mismo principio que un DAO).

12. Glosario rápido

- **APK / AAB**: paquete instalable / paquete de publicación.
- **adb**: Android Debug Bridge, la CLI para hablar con el dispositivo.
- **AVD**: dispositivo virtual (emulador).
- **BMR**: metabolismo basal (kcal en reposo).
- **Composable**: función que dibuja UI.
- **DAO**: objeto de acceso a datos (las consultas).
- **Flow**: flujo reactivo de valores.
- **KSP**: procesador de anotaciones de Kotlin (genera el código de Room).
- **MET**: equivalente metabólico (kcal/kg/hora).
- **Migración**: transformación del esquema de la base entre versiones.
- **RDA**: ingesta diaria recomendada.
- **RPE**: esfuerzo percibido (1-10).
- **SAF**: Storage Access Framework (selector de archivos del sistema).
- **Upsert**: insertar-o-actualizar.
- **Hook**: función de React (`useState`, `useEffect`...) que "engancha" estado o efectos a un componente.
- **localStorage**: almacén clave-valor del navegador, por sitio.

13. Para seguir aprendiendo

- Kotlin: kotlinlang.org/docs (oficial, excelente).
- Compose: developer.android.com/develop/ui/compose (codelabs).
- Room: developer.android.com/training/data-storage/room.
- React: react.dev/learn (tutorial oficial); Vite: vitejs.dev/guide.
- El propio código: cada pantalla de `ui/pantallas/` (app) y cada página de `web/src/paginas/` sigue el mismo patrón; leer `PantallaDiario.kt` y `web/src/paginas/Alimentos.jsx` primero es la mejor puerta de entrada.